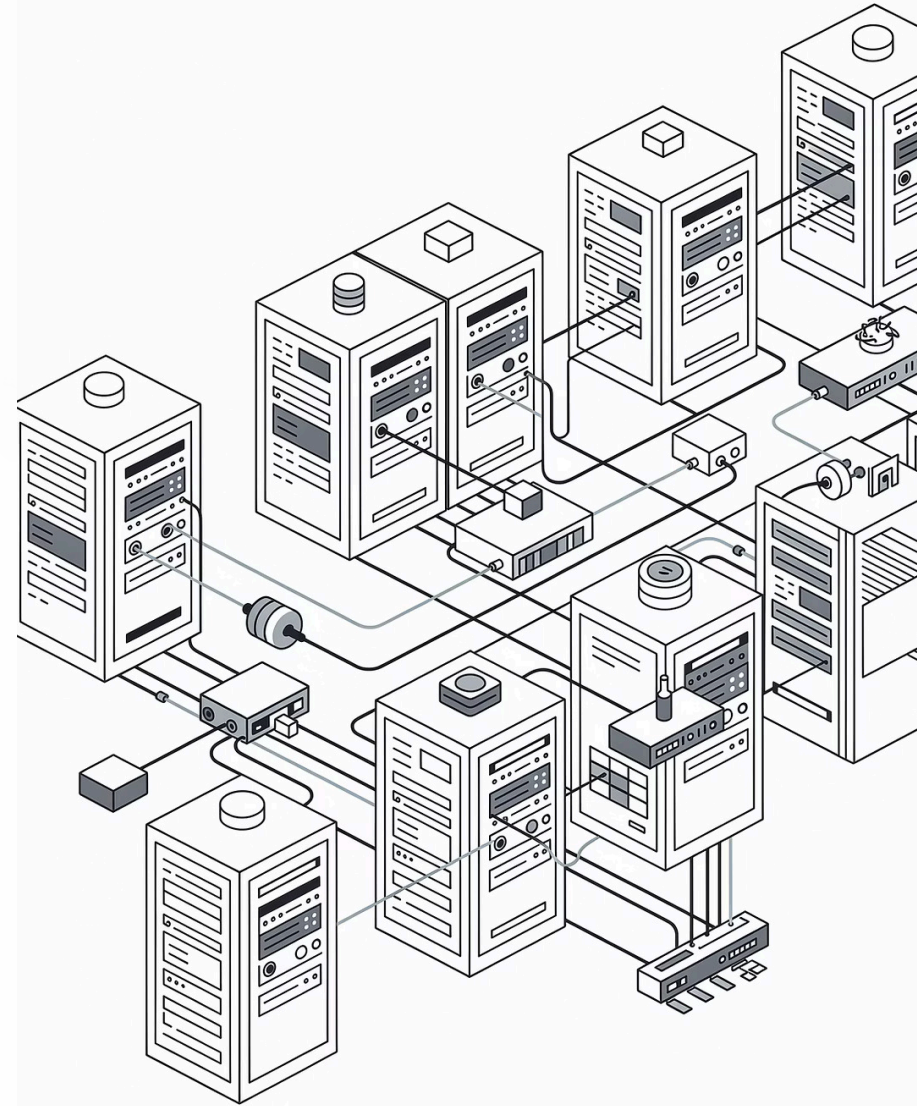


Spring : Scalabilité, Monitoring & Best Practices

Module final du parcours Spring — concevoir, opérer et faire évoluer des systèmes en production à l'échelle de l'entreprise.



Parcours de formation

Ce que nous avons déjà couvert

01

Java EE & Spring Boot

Fondamentaux web Java, configuration, injection de dépendances

03

API REST & Tests

Spring MVC, OpenAPI, tests d'intégration automatisés

02

AOP & Spring Data

Programmation transverse, transactions, accès aux données

04

Réactivité & Sécurité

Spring WebFlux, Spring Security, WebSockets

Module final

Deux grandes parties

Module 1

Scalabilité & Monitoring — construire des architectures capables de tenir la charge et d'être observées en temps réel

Module 2

Synthèse & Best Practices — consolider une vision globale de l'écosystème Spring en production

Introduction à la Scalabilité

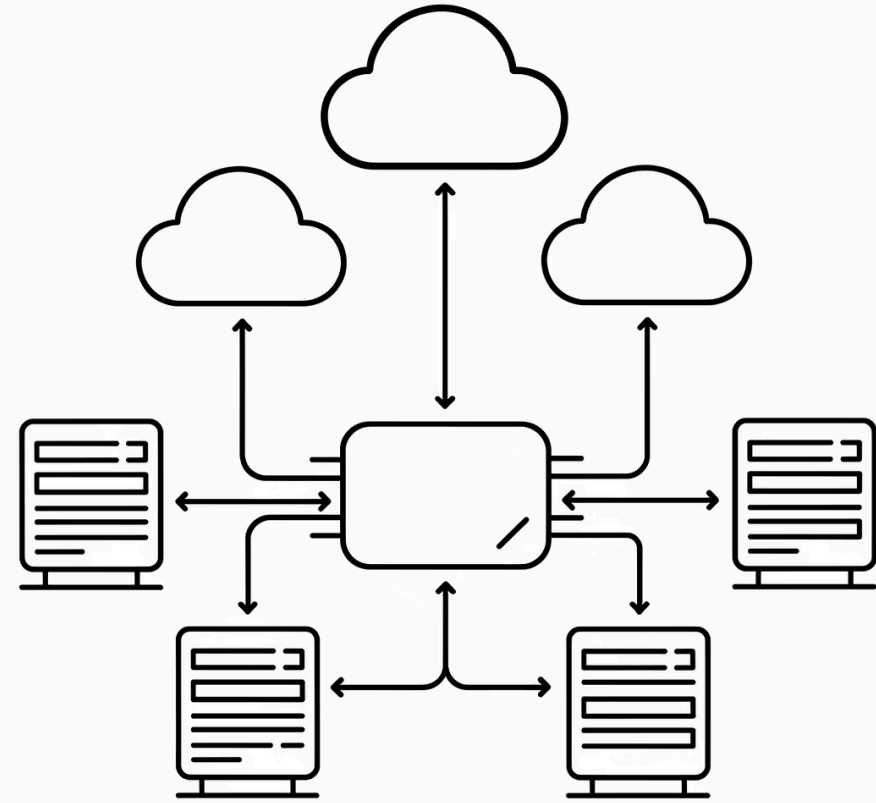
La scalabilité est la capacité d'un système à supporter une charge croissante sans dégradation des performances. Un système qui tient pour 100 utilisateurs peut s'effondrer à 10 000 si son architecture n'a pas été pensée pour évoluer.

Scalabilité verticale

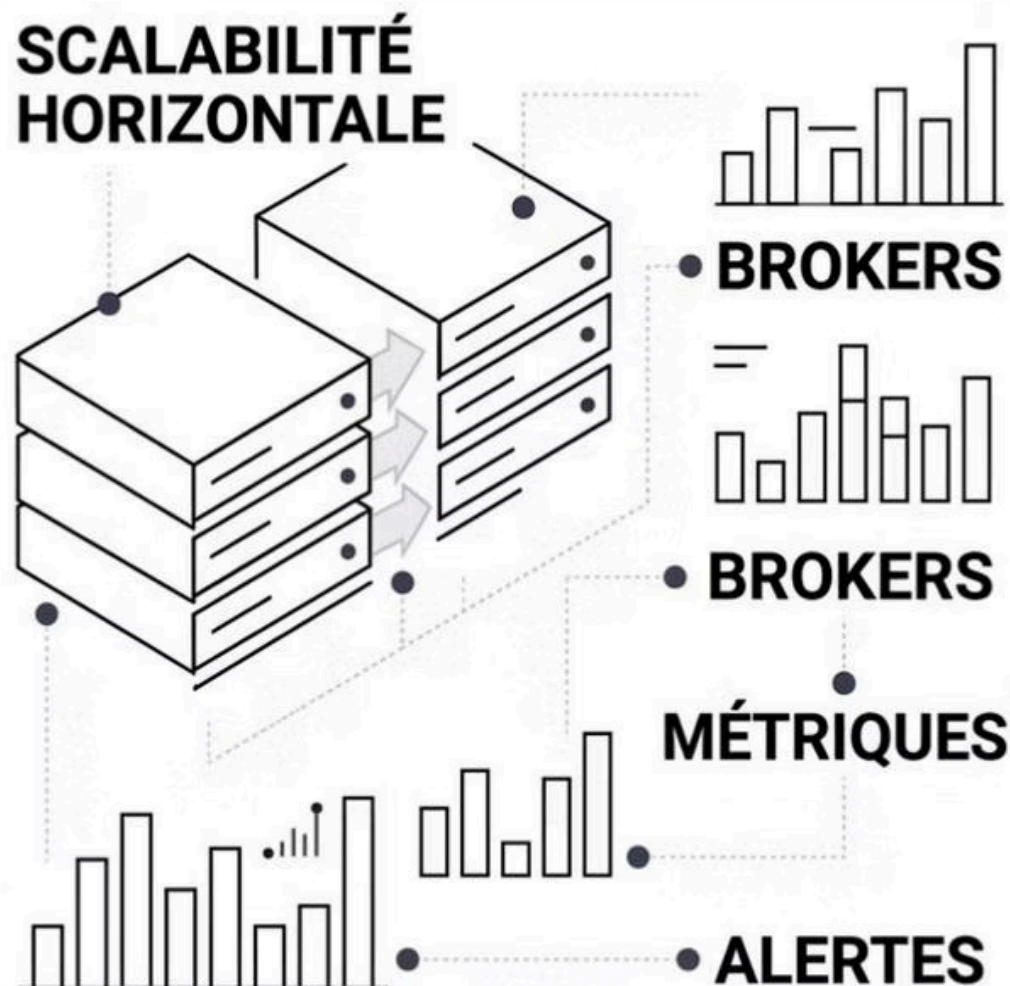
Augmenter les ressources d'un seul serveur

Scalabilité horizontale

Ajouter des instances supplémentaires — approche cloud-native privilégiée



Scalabilité Horizontale avec Spring



Principes fondamentaux

- Applications **stateless** : pas d'état en mémoire locale
- Externalisation des sessions (Redis)
- Configuration externalisée (Spring Cloud Config)
- Découverte de services (Eureka, Consul)
- Load balancing côté client (Spring Cloud LoadBalancer)

i Les sessions HTTP doivent être externalisées via Spring Session + Redis pour garantir la transparence entre instances.

Session, Cache et État Partagé



Spring Session + Redis

Externalise les sessions HTTP. Toutes les instances partagent le même état via `@EnableRedisHttpSession`.



Spring Cloud Config

Centralise la configuration dans un dépôt Git. Mise à jour dynamique via `@RefreshScope`.



Spring Cache

Abstraction via `@Cacheable`, `@CacheEvict`. Compatible Redis, Hazelcast, Caffeine.



Service Discovery

Eureka ou Consul permettent aux instances de se découvrir sans configuration statique des URLs.

Message Brokers et Clustering

Dans une architecture distribuée, les message brokers assurent un couplage faible, une livraison garantie et absorbent les pics de charge.

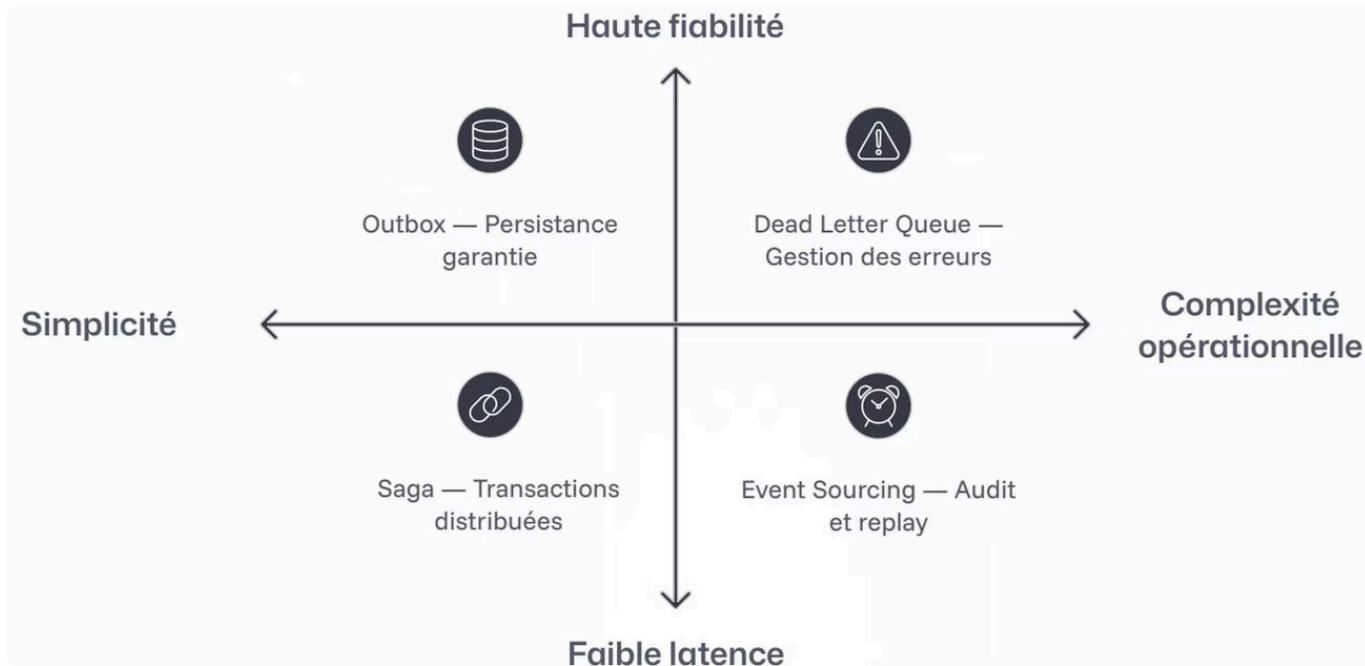
RabbitMQ – Spring AMQP

- Routage complexe (direct, topic, fanout)
- `@RabbitListener` + `RabbitTemplate`
- Dead Letter Queues pour la gestion des erreurs
- Clustering et mirroring des queues

Apache Kafka – Spring Kafka

- Flux à très haut débit, rétention des événements
- `@KafkaListener` + `KafkaTemplate`
- Consumer groups pour la scalabilité
- Exactly-once semantics en production

Patterns de Messaging en Production



Le **pattern Outbox** garantit qu'un message est publié si et seulement si la transaction en base a réussi — évitant les incohérences entre base et broker.

Outbox

Persistance garantie

Dead Letter Queue

Gestion des erreurs

Saga

Transactions distribuées

Event Sourcing

Audit et replay

Spring Boot Actuator & Micrometer

Un système en production doit être observable selon trois piliers : **métriques**, **logs** et **tracing distribué**. Actuator est le point d'entrée naturel.



Endpoints clés

`/health`, `/metrics`, `/info`, `/threaddump`



Types Micrometer

Counters, Gauges, Timers,
Distribution summaries



Métriques métier

Commandes passées, taux de succès,
durée des workflows critiques

⚠️ Sécurisez toujours les endpoints Actuator en production. N'exposez jamais `/actuator/env` sans authentification.

Stack Prometheus / Grafana

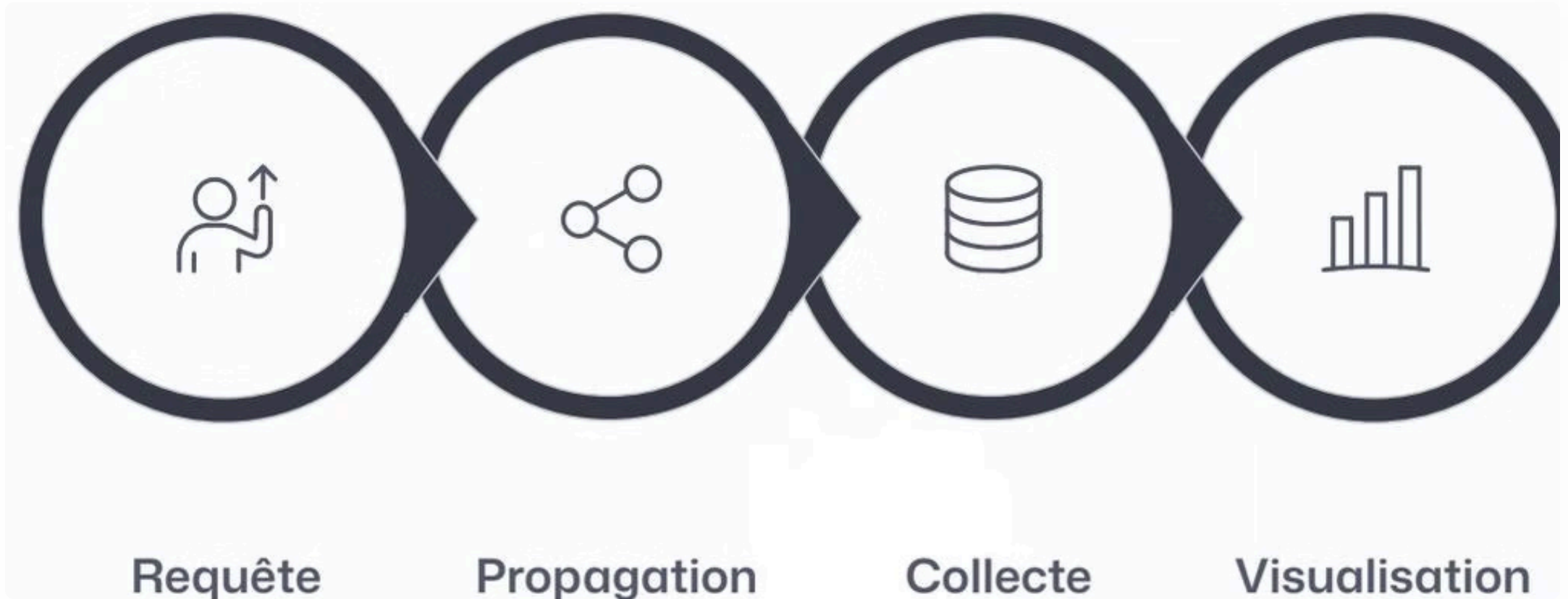
❏ Prometheus®



Prometheus collecte les métriques en mode **pull** (scraping toutes les 15s).
Grafana offre la visualisation interactive.

- Dépendance `micrometer-registry-prometheus`
- Endpoint `/actuator/prometheus` exposé
- Dashboard communautaire **#4701** "JVM Micrometer" — couvre CPU, mémoire, GC, threads, latences HTTP
- Déployable localement en quelques minutes via Docker Compose

Tracing Distribué



Micrometer Tracing (ex-Spring Cloud Sleuth) génère automatiquement des `traceld` et `spanId` injectés dans les logs via MDC, propagés dans les headers HTTP (W3C Trace Context ou B3). Les traces sont envoyées vers **Zipkin** ou **Jaeger** — configuration en quelques dépendances Maven/Gradle.

Gestion des Ressources Système



Pool JDBC (HikariCP)

Configurez `maximumPoolSize`, `connectionTimeout`, `leakDetectionThreshold`. Monitorisez `hikaricp.connections.active`.



Tuning JVM

Heap size (`-Xms`, `-Xmx`), GC algorithm (G1GC ou ZGC). Activez les GC logs en production.



Pools de threads

Tomcat (MVC) ou Netty (WebFlux). Ajustez selon le profil IO-bound vs CPU-bound.



Resilience4j

Circuit breakers, rate limiters, bulkheads. Intégration native avec Micrometer.

Alertes et Reporting

Alertmanager (Prometheus)

Routage, regroupement et mise en silence des alertes. Règles en PromQL → Slack, PagerDuty, e-mail.

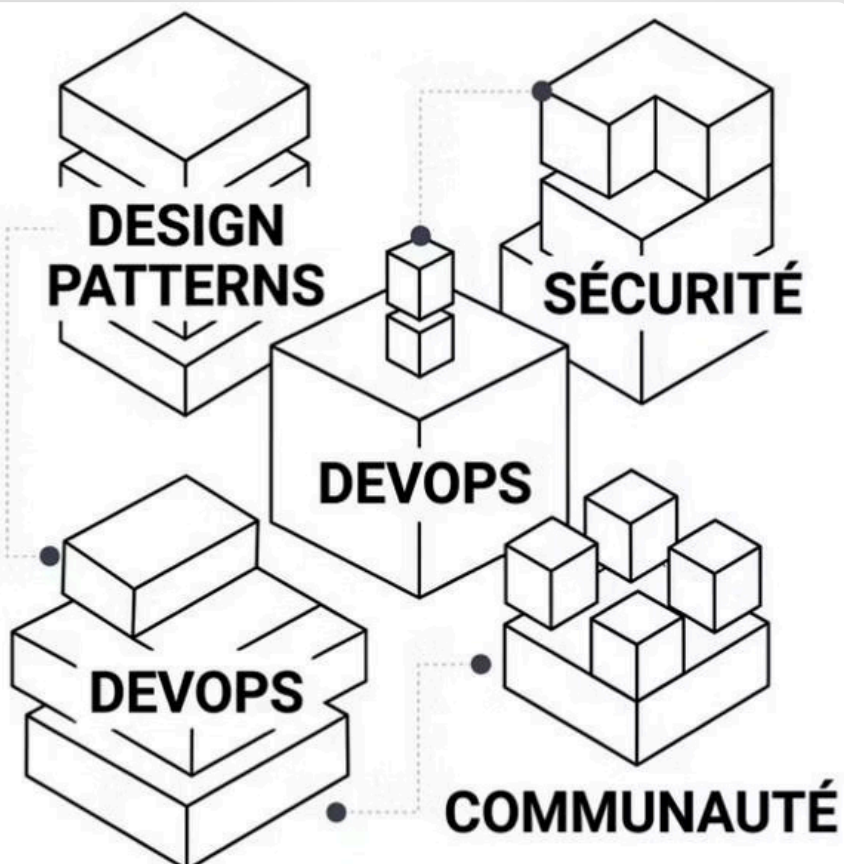
- Groupement d'alertes liées
- Inhibition des alertes redondantes
- Silences lors des maintenances

Alertes clés à mettre en place

- Taux d'erreur HTTP > 1% sur 5 minutes
- Latence P99 > seuil SLO
- Heap JVM > 80% pendant > 5 minutes
- Pool HikariCP exhausted
- Consumer lag Kafka croissant
- Circuit breaker ouvert sur service critique



Évitez l'alert fatigue : priorisez les alertes symptômes plutôt que causes.



Design Patterns Spring & Architecture

Comprendre les patterns en profondeur — pas seulement leur utilisation mécanique — distingue un développeur Spring compétent d'un architecte Spring.

Patterns Fondamentaux de Spring

IoC & Dependency Injection

Préférez l'injection par constructeur pour les dépendances obligatoires. Évitez `@Autowired` sur les champs — cela masque les dépendances et complique les tests.

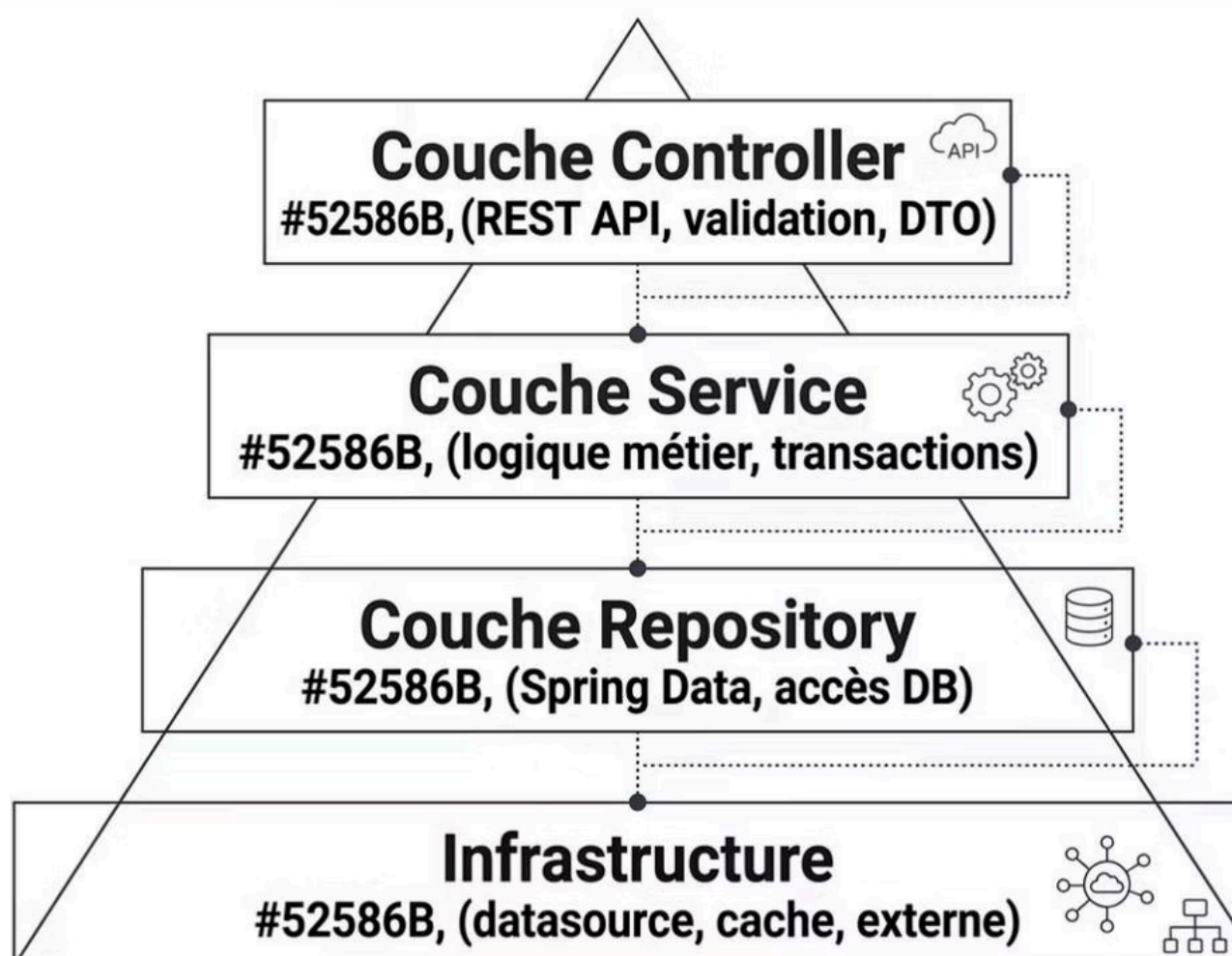
Proxy Pattern & AOP

Spring utilise des proxies JDK ou CGLIB. Les auto-appels dans la même classe contournent le proxy, rendant `@Transactional` et `@Cacheable` inopérants.

Template Method Pattern

`JdbcTemplate`, `RestTemplate`, `KafkaTemplate` encapsulent la logique répétitive tout en laissant le développeur définir la logique métier.

Architecture en Couches



Règle fondamentale

Les dépendances ne vont que vers le bas.
Un Controller peut dépendre d'un Service,
jamais l'inverse. Un Repository ne doit
jamais appeler un Service.

- **Controller** — REST API, validation, DTO
- **Service** — logique métier, transactions
- **Repository** — Spring Data, accès DB
- **Infrastructure** — datasource, cache, externe

Utilisez des **DTOs** pour isoler la couche
Controller des entités JPA.

Bonnes Pratiques de Sécurité

CSRF

Protection activée par défaut. Ne désactivez que pour les API REST stateless avec JWT/OAuth2.

Moindre privilège

Politique `deny-by-default` dans `SecurityFilterChain`. Rôles granulaires avec `@PreAuthorize`.

Secrets

Jamais de credentials en dur. Variables d'env, Spring Cloud Config chiffré, ou HashiCorp Vault.

Dépendances

`mvn dependency-check` ou Snyk en CI/CD. Maintenez Spring Boot à jour pour les correctifs de sécurité.

JWT & OAuth2 : Pièges et Bonnes Pratiques

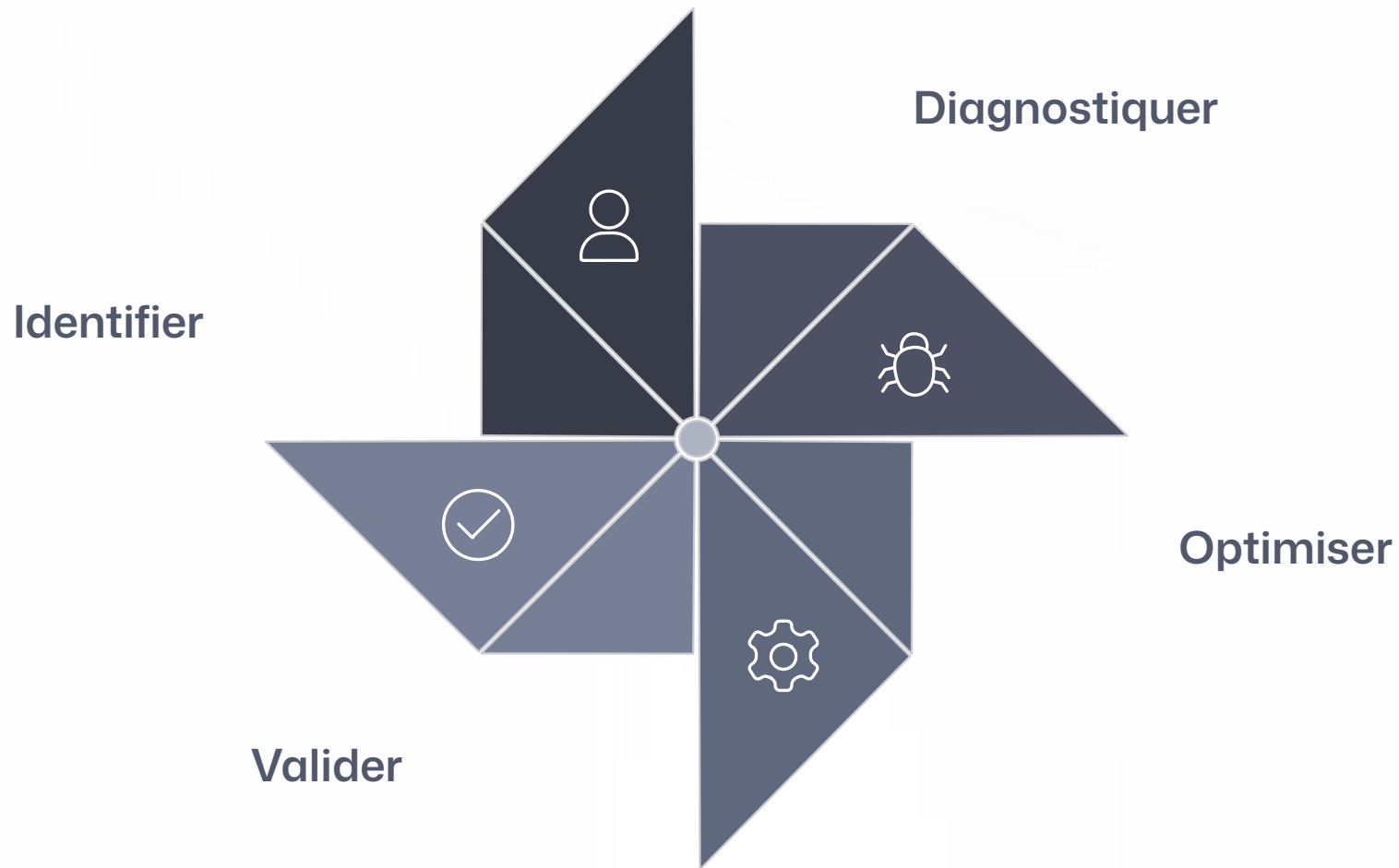
✗ Erreurs courantes

- Stocker les JWT dans localStorage (XSS)
- Algorithme none ou HS256 avec secret faible
- Ne pas valider iss et aud
- Access tokens avec durée excessive (>15 min)
- Données sensibles dans le payload JWT

✓ Meilleures pratiques

- Cookies HttpOnly + Secure + SameSite
- Algorithme RS256 ou ES256 (clés asymétriques)
- Valider iss, aud, exp systématiquement
- Access tokens courts (5-15 min) + refresh tokens
- Déléguer à un IdP robuste (Keycloak, Auth0)

Optimisation des Performances



Le problème des **N+1 queries** JPA est le plus fréquent en production. Une liste de 1000 commandes chargées en lazy génère 1001 requêtes SQL — potentiellement 100 secondes de temps de réponse.

Optimisations JPA Essentielles

→ N+1 Queries

Utilisez `@EntityGraph`, `JOIN FETCH` en JPQL, ou `@BatchSize`. Activez Hibernate Statistics en développement.

→ Cache & Indexes

Hibernate 2nd level cache pour les entités en lecture seule. Vérifiez que les colonnes de filtre (WHERE, JOIN) sont indexées.

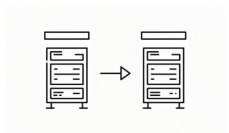
→ Projections & Pagination

Interfaces ou DTOs pour ne sélectionner que les colonnes nécessaires. Ne jamais charger une collection non bornée.

→ Read Replicas

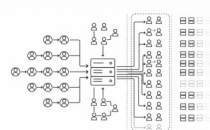
Routez les lectures vers des replicas avec Spring Data. Transactions courtes — ne maintenez pas une transaction ouverte pendant des opérations longues.

Stratégies de Déploiement



Blue/Green

Deux environnements identiques. Bascule instantanée, rollback quasi-instantané. Coût : double infrastructure.



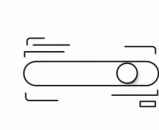
Canary Release

1-5% du trafic vers la nouvelle version. Détection précoce des régressions avec impact minimal.



Rolling Update

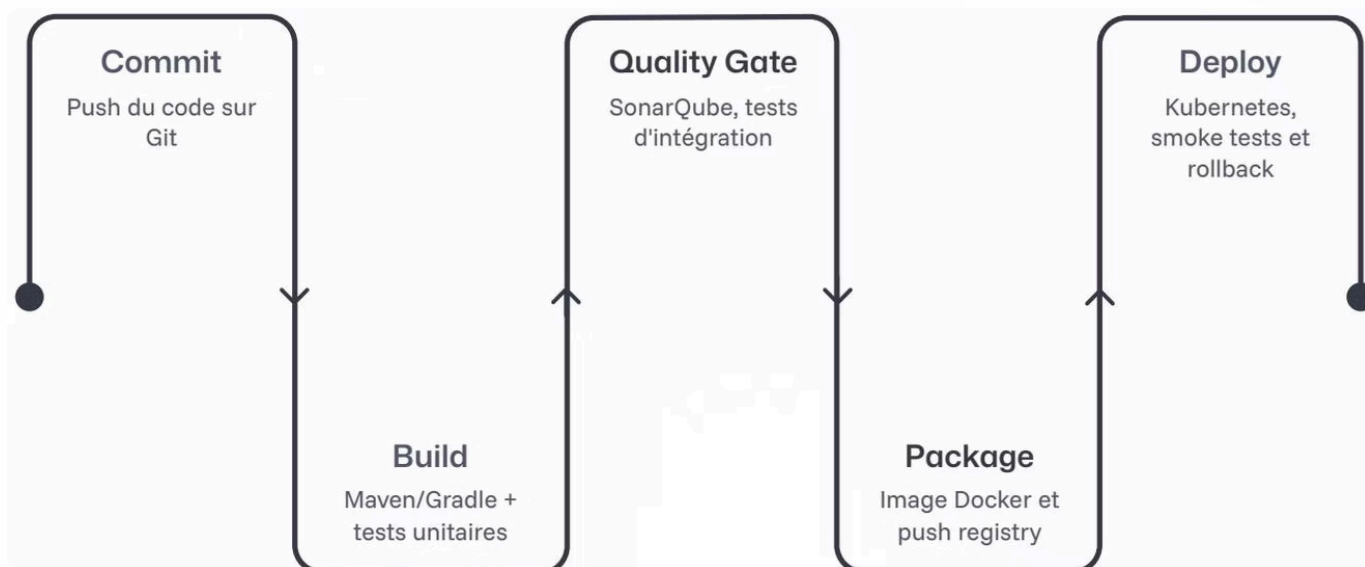
Instances mises à jour progressivement. Stratégie par défaut de Kubernetes. Nécessite la compatibilité entre deux versions simultanées.



Feature Flags

Fonctionnalités déployées mais masquées. Activation progressive via Unleash ou LaunchDarkly. Découple déploiement et livraison.

Pipeline CI/CD pour Spring Boot



Étapes clés

- **Commit** — Push sur Git
- **Build** — Maven/Gradle + tests unitaires
- **Quality Gate** — SonarQube, tests d'intégration Testcontainers
- **Package** — Image Docker + push registry
- **Deploy** — Helm/Kubernetes, smoke tests, rollback automatique

Chaque étape est une porte : si elle échoue, la pipeline s'arrête et l'équipe est notifiée.

Dockerisation & Layered JARs

Spring Boot 2.3+ supporte les JARs en couches. Combiné à un multi-stage Dockerfile, les dépendances (rarement modifiées) sont séparées du code applicatif — **un rebuild ne pousse que ~5 Mo au lieu de ~50 Mo.**

User non-root

Ne jamais faire tourner la JVM en tant que root

Image minimale

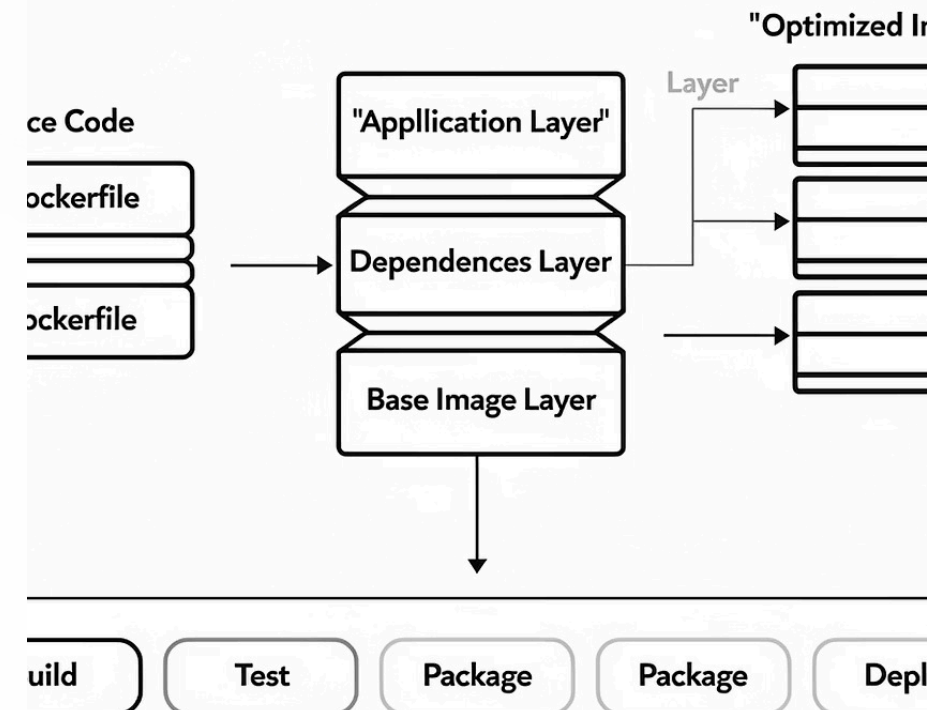
`eclipse-temurin:21-jre-alpine` plutôt qu'un JDK complet

Healthcheck

Pointant vers `/actuator/health`

JVM flags

`-XX:MaxRAMPercentage=75.0` pour respecter les limites container



Évolution & Maintenance Long Terme

Migrations BDD

Flyway ou Liquibase. Jamais de migration manuelle en production. Toujours réversibles.

Dette technique

Mesurez avec SonarQube. Réservez 20% du sprint. Quality gates automatiques en CI.

1

2

3

4

Mises à jour Spring Boot

Suivez le cycle de release. Le Spring Dependency BOM gère la compatibilité des versions.

Documentation vivante

OpenAPI auto-généré. Tests d'intégration comme documentation exécutable. ADR pour les décisions architecturales.

i Spring Boot 3.x : migration Jakarta EE 10 (`javax.*` → `jakarta.*`), Virtual Threads Java 21 (Project Loom), GraalVM Native Image — démarrage <100ms, mémoire réduite de 80%.

Ressources & Communauté Spring



Documentation Officielle

spring.io/docs — référence absolue. Lisez les Reference Guides. Spring Initializr sur **start.spring.io**.



Communauté & Open Source

GitHub, Stack Overflow (tag spring-boot), Discord officiel. Talks **SpringOne** gratuits sur YouTube.



Veille Technique

spring.io/blog, Baeldung, InfoQ, Devvxx. Newsletter **This Week in Spring** de Josh Long.



Certification Spring Professional

VMware/Broadcom — couvre Spring Core, Boot, MVC, Data, Security et tests. Préparez-vous avec les study guides officiels.